



Transformers Parameters

Last Updated : 23 Jul, 2025

The parameters of a transformer model fundamentally shape its power, efficiency, and overall behavior. These parameters define the model's composition, capacity, and learning ability. Unlike traditional RNN and CNN, transformers are completely dependent on the attention mechanism to attract global dependence between input and output. In this article, we will discover these parameters deeply, understand their significance, and see how they are used in the real world.

Types of Transformer Parameters

1. Structural Parameters

These define the architecture and capacity of the Transformer model.

Parameter	Description
num_hidden_layers	Number of encoder/decoder blocks in the model
hidden_size (d_model)	Dimensionality of token embeddings and hidden states
num_attention_heads	Number of attention heads for multi-head self-attention
intermediate_size	Size of feed-forward hidden layer
vocab_size	Number of tokens in the vocabulary
max_position_embeddings	Maximum sequence length model can handle

2. Training/Regularization Parameters

These affect how the model learns during training and help prevent overfitting.

Parameter	Description
dropout_rate / attention_probs_dropout_prob	Dropout applied to attention probabilities and outputs
hidden_dropout_prob	Dropout applied to hidden layers
initializer_range	Range for random weight initialization
layer_norm_eps	Small epsilon added for numerical stability during layer normalization

3. Functional/Behavioral Parameters

These define how certain operations behave inside the model.

Parameter	Description
activation_function	Activation used in feed-forward layers (relu, gelu, etc.)
attention_type (in some custom models)	Defines attention mechanism (e.g., global/local)
type_vocab_size	Used for sentence classification tasks (e.g., next sentence prediction)

Understand core transformer parameters

1. Num_layers (or num_hidinden_layers)

This parameter defines the maximum sequence length the model is capable of processing Each layer has a self -emission mechanism and a forward nerve network. Increase in the number of layers often leads to better performance, but also increases calculation costs.

2. D_model (or hidde_size)

D_model represents the shape of built-in and hidden conditions. In Burt Base there is 768. A higher value of D_{model} enables the model to learn more complex representations, but it also increases memory consumption.

3. Num_attnon_heads

Focus with several heads shares the input into several major subjects, so that the model can focus on different parts of the sentence at the same time. The total hidden form should be divisible with the number of heads.

4. D_ff (intermediate_ses)

This defines the alphabet of the stranger layer in each coder block. Usually it is larger than the D_model (eg, the 3072 in the Burt base for D_model of 768).

5. Dropout_rite

Dropout used to regulate overfeating. These are randomly selected units from the network that are dropped during the training process

6. Vocab_sise

This indicates that unique token can understand models. It directly affects the size of the input and output that involves layers.

7. Max_position_mbeddings

Transformers do not naturally understand the order, so the condition involves added. This parameter determines the maximum sequence length that the model can process.

8. Activation_function

Transformers usually use relay or gallu as a non-related activation in foreign layers. GELU is often preferred in models like BERT and GPT due to its smooth activation curve, which enhances learning stability and performance

This parameter controls the distribution of initial loads. The small value as 0.02 ensures stable learning in the initial stages.

9. Layer_norm_eps

An appsillon value to avoid zero division in team normalization.

Code to Explore Parameters

Using the Hugging Face Transformers library, you can easily inspect and modify parameters:

```
from transformers import BertConfig, BertModel

# Load the default BERT configuration
config = BertConfig()
print(config)
```

Output:

```
BertConfig {
  "attention_probs_dropout_prob": 0.1,
  "hidden_act": "gelu",
  "hidden_size": 768,
  "initializer_range": 0.02,
  ...
}
```

Use of the real world of parameters

In scenarios in the real world, these parameters are important:

- Reducing **num_layers** or **hidden_size** helps when distributing the model on the age unit.
- Increasing **NUM_ATION_HEADS** or **intermediate_SES** helps capture complex patterns in large datasets.
- Adjusting **dropout_rett**, depending on the data set size, helps to avoid overfitting.

Similar Articles

- [Introduction to Transformer Models](#)
- [Multi-Head Attention Explained](#)
- [BERT Implementation Using Hugging Face](#)

Comment

V verma...

9

Article Tags:

NLP

AI-ML-DS

Deep Learning

 **Corporate & Communications Address:**
A-143, 6th Floor, Sovereign Corporate
Tower, Sector- 136, Noida, Uttar
Pradesh (201305)

 **Registered Address:**
K 061, Tower K, Gulshan Vivante
Apartment, Sector 137, Noida, Gautam
Buddh Nagar, Uttar Pradesh, 201305



Legal	Practice	Programming	ML and Data	Interview
Privacy	Problems	Languages	Science	Corner
Policy	Blogs	DSA	DSA and	Aptitude
Careers	Upskill	Web	Placements	Puzzles
Contact Us	Courses	Technology	Web	GfG 160
Corporate	Connect	AI, ML &	Development	System Design
Solution		Data Science	Data Science	
Campus		DevOps	Programming	
Training		CS Core	Languages	
		Subjects	DevOps &	
		GATE	Cloud	
		School	GATE	
		Subjects	MongoDB	
		Software and	Certifications	
		Tools		